Hardware-Software Co-Design of a Multiplier-Less SiLU Custom Instruction for RISC-V Edge AI Accelerators

Course: BAEVD202 COMPUTER ARCHITECTURE
Slot: A1+TA1+TAA1

Overview

This project executes a comprehensive Hardware-Software Co-Design methodology to alleviate the computational bottlenecks associated with complex, non-linear activation functions in edge AI models. Aligning with contemporary architectural trends (Modules 3 and 5), the project replaces resource-heavy digital multipliers (DSPs) with a lightweight, piecewise shift-and-add arithmetic unit designed to approximate the Sigmoid Linear Unit (SiLU). The hardware datapath is synthesized on an Intel Cyclone IV FPGA via Quartus to extract physical metrics such as critical path latency and zero-DSP utilization. To evaluate system-level integration, this multiplier-less logic is mapped natively into the RISC-V Instruction Set Architecture (ISA) as a standard custom opcode (`custom-0`) inside the gem5 architectural simulator (Syscall Emulation mode). By executing cross-compiled, cycle-accurate C micro-benchmarks via inline assembly, the pipeline definitively measures the Power-Performance-Area (PPA) advantages of tightly coupled custom instructions over traditional software math libraries for AI inference.

Team Details

1) VRINDA PANDEY - 25BVD0126
2) AYUSH KRISHNA - 25BVD0112
3) TANVEE CHINTALA - 25BVD086
4) CHANDNI SRIVASTAVA - 25BVD0107
5) ESWAR SESHA SAI - 25BVD0127

Project Timeline and Milestones

Milestone 1: Hardware Synthesis and Cross-Compilation Pipeline
Target: One week before CAT1 Review
Finalize the Verilog implementation of the piecewise shift-and-add SiLU approximation. Synthesize the logic using Intel Quartus (targeting the Cyclone IV FPGA) to extract critical path latency and confirm zero DSP block utilization. Concurrently, establish the Docker-hosted gem5 environment and successfully cross-compile a baseline C AI mathematical micro-kernel (using standard math libraries) for the bare-metal RISC-V instruction set.

Milestone 2: ISA Modification and System Integration
Target: One week before CAT2 Review
Modify the gem5 RISC-V `decoder.isa` file to define the new `custom-0` opcode, injecting the C++ logic that mirrors the hardware shift-and-add arithmetic. Hardcode the physical latency metrics extracted from Quartus into the simulator's timing model to ensure cycle-accurate hardware simulation. Update the C micro-benchmark to utilize inline RISC-V assembly, successfully triggering the `custom-0` instruction during a Syscall Emulation (SE) mode simulation run.

Milestone 3: Benchmark Evaluation and Final Deliverables
Target: One week before FAT Final Review
Execute rigorous A/B benchmark testing within gem5, comparing the total execution cycles and memory fetch operations of the baseline software implementation against the accelerated `custom-0` implementation. Extract and format the resulting data from the gem5 statistics logs. Compile a professional architectural analysis report quantifying the system-level speedup and hardware efficiency gains for course demonstration.

Expected Outcomes and Key Deliverables

- A fully synthesized Verilog module demonstrating a multiplier-less SiLU approximation with verifiable zero-DSP utilization on an Intel Cyclone IV FPGA.
- A modified, cycle-accurate gem5-RISCV simulation environment featuring a custom ISA extension (`custom-0`) calibrated with physical FPGA latency metrics.
- Cross-compiled C/C++ AI micro-benchmarks utilizing inline RISC-V assembly to bypass standard software math libraries.
- Comprehensive statistical analysis (extracted from gem5 `stats.txt`) proving the reduction in total execution cycles and memory accesses achieved via tight hardware coupling.
- Technical documentation, architectural analysis, and presentation materials for course review, demonstrating a complete end-to-end Hardware-Software Co-Design pipeline.